

Assignment 1

Computational Methods for Network Science

NETS/PHYS 7052, Fall 2026

Haley Broadus https://github.com/Mangbunn/nets7052_fa26

1. Question 1.

- (a) Each format was saved and reloaded using default arguments only. Table I shows the results. The original graph G has 1490 nodes and 19,025 edges, is directed, and has node attributes.

Format	Size (KB)	Nodes	Edges	Directed?	Attrs?	graphs_equal?
Edge list	213.58	1224	16718	False	False	False
GraphML	795.55	1490	19025	True	True	False
JSON	810.51	1490	19025	True	True	True
Pickle	448.64	1490	19025	True	True	True
Dense adj. matrix	17344.66	1490	16718	False	False	False

TABLE I. Round-trip results with default arguments.

- (b) JSON and pickle match G in every column, so there is nothing to investigate. (JSON stores the **directed** flag in the file itself.)

Edge list. G_{back} has 1224 nodes instead of 1490, 16718 edges instead of 19025, is undirected, and has no node attributes. Node labels are strings in both graphs.

- *Missing nodes (save-side).* Every edge appears in the file, so the $1490 - 1224 = 266$ missing nodes must have no incident edges. The writer only outputs edges, so isolated nodes are never written.
- *Missing node attributes (save-side).* Each line of the file has the form (source) (target) {edge attributes}. There is no node data in the file.
- *Direction and edge count (load-side).* The file lists each edge as (source, target), so the direction is recorded. `read_edgelist` defaults to an undirected **Graph**, which merges reciprocal links ($A \rightarrow B$ and $B \rightarrow A$), so the edge count drops by $19025 - 16718 = 2307$.

Dense adjacency matrix. G_{back} has all 1490 nodes (the matrix dimension fixes the node set), but 16718 edges, is undirected, and has no attributes. Loading the `.numpy` file gives a 1490×1490 array of plain numbers.

- *Labels and attributes (save-side).* `to_numpy_array` replaces the string IDs by row/column positions and discards all node data. The reloaded nodes are integers $0, \dots, 1489$, while the originals were strings.
- *Direction and edge count (load-side).* The array is asymmetric, so the direction is still encoded in it, but `from_numpy_array` defaults to an undirected graph, merging reciprocal links as above.

- *Edge weights.* `from_numpy_array` also adds a `weight` attribute to every edge, which the original did not have.

GraphML. Nodes, edges, direction and node attributes all match. The only difference is discussed in part (d).

- (c) **Edge list.** `create_using=nx.DiGraph` restores the direction and the 19,025 edges. The missing isolated nodes and all node attributes cannot be recovered with any argument, because the format has no place to store node data. They must be restored from the original TSV file, e.g. with `G.back.add_nodes_from(...)`, passing each node's `website` and `political_orientation`.

GraphML. No argument changes how `read_graphml` builds the graph-level defaults. We can delete them after loading:

```
del G_back_graphml.graph['node_default']
del G_back_graphml.graph['edge_default']
```

Dense adjacency matrix. `create_using=nx.DiGraph` restores the direction and the edge count, and `edge_attr=None` removes the added `weight` attribute. At save time, `nodelist=list(G.nodes())` pins down the row/column order. The format cannot store labels, so the node list must be saved separately (a JSON or pickle side file, or the original TSV). With that list we relabel:

```
mapping = dict(enumerate(nodelist))
G_back_matrix = nx.relabel_nodes(G_back_matrix, mapping)
```

The node attributes also have to come from the original TSV file.

- (d) `graphs_equal` compares the graph-level attribute dictionary `G.graph` as well as nodes, edges and their attributes. For G this dictionary is `{}`, while the reloaded GraphML graph has `{'node_default': {}, 'edge_default': {}}`. When `read_graphml` parses the `<key>` declarations, it records default values for each declared attribute, even when no `<default>` is given. This is an artifact of the parser, and no information about the graph is lost, so I consider it harmless. After deleting both keys, `graphs_equal` returns `True`.
- (e) Both formats turn integer labels into strings. Passing `nodetype=int` to `read_edgelist` and `node_type=int` to `read_graphml` returns integers. Node IDs carry no type in either file, so a colleague would have to be told that the labels are integers (verbally or in documentation). GraphML does tag the types of node and edge *attributes* in its `<key>` declarations, but not of the node IDs.
- (f) The largest file is the dense adjacency matrix and the smallest is the edge list. The edge list stores only the edges that exist, and no isolated nodes or node attributes. The dense matrix stores one entry for every *possible* edge, $1490 \times 1490 = 2,220,100$ entries, whereas the edge list stores only 19,025 edges. The matrix dimension also fixes the node set, which is why it keeps isolated nodes while the edge list does not.

Average degree: $\bar{d} = |E|/|N| = 19025/1490 \approx 12.8$. For 10^6 nodes, $|E| \approx 12.8 \times 10^6$.

Edge list. Bytes per edge: $\frac{213.58 \times 1024}{19025} \approx 11.5$. Then

$$11.5 \frac{\text{bytes}}{\text{edge}} \times 12.8 \times 10^6 \text{ edges} \approx 1.5 \times 10^8 \text{ bytes } (\approx 150 \text{ MB}).$$

Dense matrix. Bytes per entry: $\frac{17,344.66 \times 1024}{1490^2} \approx 8$. Then

$$8 \frac{\text{bytes}}{\text{entry}} \times (10^6)^2 \text{ entries} = 8 \times 10^{12} \text{ bytes} \approx 8 \text{ TB}.$$

(g) The dangerous line is

```
G_back_pickle = pickle.load(fp)
```

Pickle does not just store data. Loading a pickle file runs instructions that rebuild the object, and pickle cannot tell harmless instructions from malicious ones. If a colleague sends you a `.pkl` file and you call `pickle.load()`, you are effectively running a program they wrote, with all the permissions of your account. Only unpickle files from sources you trust.

2. Question 2.

- (a) The code breaks when $i = u, v$ or $j = u, v$. Or when the edges (i, v) or (j, u) are already present. Thus, when $N \leq 3$, we do not have enough nodes available to satisfy condition one. Furthermore, (thanks to the help of Wolfram MathWorld;) any graph that is characterized by the **forbidden** induced subgraphs:

$$C_4, P_4, 2P_2$$

is not two-switchable (YIPIII)[1]. This includes the complete graph, or its complement, but also some other fun things like the full house graph and the claw graph. One can clearly see that, for the complete graph, the unswitchable nature comes from the fact that (i, v) and (j, u) are already present in the graph by construction. For graphs such as a star graph, the unswitchability comes from the fact that all nodes are incident to the same center node (and not incident to any other nodes) thus, we cannot satisfy condition 1.

A 2-switch takes two independent edges, deletes them, and replaces them with a different pairing of the same 4 endpoints. For this swap to be legal and change the layout of the graph, it requires a very specific setup among those 4 vertices (1) the original 2 edges must exist, (2) the potential 2 replacement edges must not already exist. The absence of any P_4, C_4 , or $2P_2$ subgraph implies that condition 2 is not possible.

One should note that the $2P_2, C_4, P_4$ graphs are allowed by this algorithm, but are trivially two-switchable as it will only produce graphs that are isomorphic to

the original graph.

Algorithm 1 Edge Exchange with Failure Cutoff

(b) **procedure** EDGEEXCHANGE((G, t, max_fail))

```

   $s \leftarrow 0$ 
   $fail \leftarrow 0$ 
  while  $s < t$  and  $fail < max\_fail$  do
     $(i, j) \leftarrow \text{GETRANDOMEDGE}(G)$ 
     $(u, v) \leftarrow \text{GETRANDOMEDGE}(G)$ 
    if  $i = u$  or  $i = v$  or  $j = u$  or  $j = v$  then
       $fail \leftarrow fail + 1$ 
      continue
    end if
    if  $\text{HASEDGE}(G, i, v)$  or  $\text{HASEDGE}(G, j, u)$  then
       $fail \leftarrow fail + 1$ 
      continue
    end if
    REMOVEEDGE( $G, i, j$ )
    REMOVEEDGE( $G, u, v$ )
    ADDEDGE( $G, i, v$ )
    ADDEDGE( $G, j, u$ )
     $s \leftarrow s + 1$ 
     $fail \leftarrow 0$ 
  end while
  return  $G$ 
end procedure

```

The fail counter resets after a successful attempt because it is trying to measure how many attempts IN A ROW have failed. It's normal for a graph to experience some failures, and we don't want to terminate early because the fail counter hit its max too soon. By letting it reset, we're letting the algorithm detect that there have been no changes for a long time and should terminate. Note that this does not address the isomorphic issue mentioned earlier. This algorithm will use all s attempts on a $C_4, P_4, 2P_2$ graph just switching back and forth between node relabelings.

- (c) First, let's investigate how it's possible for an edge swap to change connectivity! Let G be a simple graph, and suppose we are about to attempt the swap

$$(i, j), (u, v) \longrightarrow (i, v), (j, u).$$

Define

$$G' = G - \{(i, j), (u, v)\}, \quad G'' = G' + \{(i, v), (j, u)\}.$$

G' is the graph with both edges removed (the intermediate state), and G'' is the

graph after the full swap is applied. When does the swap change the number of components?

1) If, in G , there is no path from i to u (equivalently: (i, j) and (u, v) lie in different components C_1, C_2), then after the swap the new edges (i, v) and (j, u) each cross between C_1 and C_2 . So in G'' a path $i \rightsquigarrow u$ (and $i \rightsquigarrow j$) now exists. The two components merge:

$$\# \text{components}(G'') = \begin{cases} \# \text{components}(G) - 1 & \text{if } i \rightsquigarrow j \text{ in } G'' \\ \# \text{components}(G) & \text{otherwise.} \end{cases}$$

2) If, in G , a path from i to u does exist (so (i, j) and (u, v) lie in the same component), the swap can disconnect that component (but only if (i, j) and (u, v) together formed a 2-edge cut separating i from j or u from v , so that removing both edges severs the component, and neither replacement edge (i, v) or (j, u) reconnects it.) We detect this by checking, in G'' , whether a path from i to j still exists:

$$\# \text{components}(G'') = \begin{cases} \# \text{components}(G) + 1 & \text{if } i \not\rightsquigarrow j \text{ in } G'' \\ \# \text{components}(G) & \text{otherwise.} \end{cases}$$

WLOG note: These two checks cover all cases up to relabeling: which of the four vertices is called i, j, u, v is arbitrary, so any merge or split is caught by one of these two queries after suitable relabeling. Only one check is ever relevant per attempt: the merge check only matters when i, j, u, v start in different components, and the split check only matters when they start in the same one.

Given that the structural checks already passed, the induced subgraph on $\{i, j, u, v\}$ can only be one of $2P_2$, P_4 , or C_4 (the edges (i, v) , (j, u) are excluded by construction, leaving only (i, u) and (j, v) as possible extra edges). If (i, u) is already present, then this edge survives the swap untouched, and together with the new edge (j, u) it forms the path $i-u-j$ in G'' — so $i \rightsquigarrow j$ holds regardless of the rest of the graph's structure. Symmetrically, if (j, v) is present, the surviving edge (j, v) plus the new edge (i, v) gives the path $i-v-j$. The same surviving edges also already connect i to u in G itself, so the merge case is ruled out too. Hence:

$$\text{induced subgraph on } \{i, j, u, v\} \in \{P_4, C_4\} \implies \# \text{components}(G'') = \# \text{components}(G),$$

provably, with a single $O(1)$ edge lookup and no search at all. By contrapositive, a change in component count (merge or split) can only occur when the induced subgraph is $2P_2$, i.e. when neither (i, u) nor (j, v) is present. In that case there is no local shortcut, and we fall back to the path checks above, run directly.

Algorithm 2 Edge Exchange Preserving Connectivity

procedure EDGEEXCHANGESAMECONNECTIVITY(G, t, max_fail)

 $s \leftarrow 0$
 $fail \leftarrow 0$
while $s < t$ **and** $fail < max_fail$ **do**
 $(i, j) \leftarrow \text{GETRANDOMEDGE}(G)$
 $(u, v) \leftarrow \text{GETRANDOMEDGE}(G)$
if $i = u$ **or** $i = v$ **or** $j = u$ **or** $j = v$ **then**
 $fail \leftarrow fail + 1$
continue
end if
if $\text{HASEDGE}(G, i, v)$ **or** $\text{HASEDGE}(G, j, u)$ **then**
 $fail \leftarrow fail + 1$
continue
end if
 $\triangleright$ connectivity check

if $\text{HASEDGE}(G, i, u)$ **or** $\text{HASEDGE}(G, j, v)$ **then**
 $\triangleright P_4$ or $C_4 \rightarrow$ provably safe, $O(1)$, no search needed

else
 $\triangleright 2P_2 \rightarrow$ must determine outcome via search

if not $\text{PATH EXISTS}(G, i, u)$ **then**
 $G_1 \leftarrow \text{REMOVEEDGE}(G, i, j)$
 $G_2 \leftarrow \text{REMOVEEDGE}(G_1, u, v)$
 $G_3 \leftarrow \text{ADDEDGE}(G_2, i, v)$
 $G'' \leftarrow \text{ADDEDGE}(G_3, j, u)$
if $\text{PATH EXISTS}(G'', i, j)$ **then**
 $fail \leftarrow fail + 1$
 $\triangleright$ different components $\rightarrow$ MERGE $\rightarrow$ forbidden

continue
end if
 $G_1 \leftarrow \text{REMOVEEDGE}(G, i, j)$
 $G_2 \leftarrow \text{REMOVEEDGE}(G_1, u, v)$
 $G_3 \leftarrow \text{ADDEDGE}(G_2, i, v)$
 $G'' \leftarrow \text{ADDEDGE}(G_3, j, u)$
if not $\text{PATH EXISTS}(G'', i, j)$ **then**
 $fail \leftarrow fail + 1$
 $\triangleright$ SPLIT $\rightarrow$ forbidden

continue
end if
end if
 $\text{REMOVEEDGE}(G, i, j)$
 $\text{REMOVEEDGE}(G, u, v)$
 $\text{ADDEDGE}(G, i, v)$
 $\text{ADDEDGE}(G, j, u)$
 $s \leftarrow s + 1$
 $fail \leftarrow 0$
return G

The algorithm starts with the cheap local check (is the induced subgraph on $\{i, j, u, v\}$ a P_4/C_4 , provably safe with one edge lookup), and only if that fails, the two expensive global checks (is i already connected to u ? and, if so, does removing $(i, j), (u, v)$ and adding $(i, v), (j, u)$ still leave i connected to j ?). Each of those checks needs its own explicit branch, its own early exit on failure, and its own explicit reconstruction of G'' . The result is a procedure conceptually simple but computationally expensive, due to the nature of a BFS to find if a path between nodes exists. (I don't think I came up with a very efficient algorithm lol)

- (d) Consider 1 and -1 to be colors for the nodes of a graph; then we can say that the edges are colored either (1,1), (-1,-1), or (-1,1). Let $E_{1,1}, E_{-1,-1}, E_{-1,1}$ be the respective sets of edges of each color class. Our goal after each swap is to maintain the size of each set. We can do this by simply checking the edge sets before and after swapping. (Linear Time Complexity :))

Algorithm 3 Edge Exchange Preserving Attribute Correlation

procedure EDGEEXCHANGEATTRIBUTEPRESERVING(G, x, t, max_fail)

$s \leftarrow 0$

$fail \leftarrow 0$

while $s < t$ **and** $fail < max_fail$ **do**

$(i, j) \leftarrow \text{GETRANDOMEDGE}(G)$

$(u, v) \leftarrow \text{GETRANDOMEDGE}(G)$

if $i = u$ **or** $i = v$ **or** $j = u$ **or** $j = v$ **then**

$fail \leftarrow fail + 1$

continue

end if

if $\text{HASEDGE}(G, i, v)$ **or** $\text{HASEDGE}(G, j, u)$ **then**

$fail \leftarrow fail + 1$

continue

end if

$before \leftarrow \{\text{COLOR}(x_i, x_j), \text{COLOR}(x_u, x_v)\}$

$after \leftarrow \{\text{COLOR}(x_i, x_v), \text{COLOR}(x_j, x_u)\}$

if $before \neq after$ **then**

$fail \leftarrow fail + 1$

continue

end if

$\text{REMOVEEDGE}(G, i, j)$

$\text{REMOVEEDGE}(G, u, v)$

$\text{ADDEEDGE}(G, i, v)$

$\text{ADDEEDGE}(G, j, u)$

$s \leftarrow s + 1$

$fail \leftarrow 0$

end while

return G

end procedure=0

▷ attribute check

This constraint does not make the algorithm more complex, but it does severely

limit the number of acceptable swaps one can make.

Let $|E_{1,1}|, |E_{-1,-1}|, |E_{-1,1}|$ be the size of the respective edge color sets. Define the proportions:

$$p_{++} = |E_{1,1}|/|E|, p_{--} = |E_{-1,-1}|/|E|, p_{+-} = |E_{-1,1}|/|E|$$

. Note that $p_{++} + p_{--} + p_{+-} = 1$. Since the algorithm picks “direction” uniformly, we can construct a table with e1 color, e2 color and the fraction of combinations that preserve the size of the color sets.

e_1 color	e_2 color	fraction of directions preserving color multiset
++	++	1
++	--	0
++	+-	1
--	--	1
--	+-	1
+-	+-	$\frac{1}{2}$

TABLE II. Fraction of the two possible swap directions that preserve the color multiset $\{m_{++}, m_{--}, m_{+-}\}$, for each combination of edge colors. Since $p_{++} + p_{--} + p_{+-} = 1$ and the algorithm picks a direction uniformly at random, these fractions directly give the conditional probability of validity for each color pairing.

$$P(valid) = p_{++}^2 + p_{--}^2 + \frac{1}{2}p_{+-} + 2p_{++}p_{+-} + 2p_{--}p_{+-}$$

Remember that $p_{++} + p_{--} + p_{+-} = 1$

$$\implies 1 - P(valid) = 2p_{++}p_{--} + \frac{1}{2}p_{+-}^2$$

Thus, when the system is homogenous, $P(valid) = 1$, When the system is all (+-)-edge color $P(valid) = 1/2$, and when the system is roughly balanced $p_{++} = p_{--} = p_{+-} = 1/3$ we get that $1 - P(valid) = 5/18$.

- (e) s counts successful swaps applied to G , and the loop guarantees exactly t changes to the graph. If s were incremented on every attempt, the loop would terminate once t attempts have been made, regardless of how many of them actually modified G . Thus, the final graph you would get would have fewer changes applied to it compared to the original graph. And when we consider more restrictive algorithms like in c / d, we could potentially see almost no changes before the algorithm terminates. Once again, this algorithm does not consider graph isomorphisms, so especially for smaller graphs, or more restrictive algorithms, it is entirely possible that the final graph outputted is actually the same graph as the input but with a relabeling of the nodes, and that would make for a very bad null model!

3. Question 3.

- (a) (c): Our results are identical for two main reasons. We are both working on simple, unweighted, and undirected graphs. This helps ensure that we are measuring the same thing (the number of closed walks they participate in). Currently, we both have the same guardrails implemented. BUT while testing, I didn't realize I forgot to deal with the weighted network case and only noticed when seeing the comparison for the karate club graph. My implementation specifically used `eigh` to find the eigenmodes, so my results were completely different from `networkx`! However, I then changed it to convert weighted graphs to unweighted graphs, and both our values matched again, so I'm assuming they've implemented a similar method for dealing with weighted graphs.

Furthermore, since the SGC has a very simple (and fun) mathematical expression, it's very easy for us to implement the same formulas, and if we're both using the same formulas on the same graphs, then we should obviously get the same values. (All the calculations are less than 5 lines of code, so there's not much room for rounding errors to accumulate)

- (b) I did several test cases. For the first section, I wanted to make sure my error cases raised correctly, so I chose the following:
- i. ("not a graph", [1, 2, 3], `TypeError`),
 - ii. ("empty graph", `nx.empty_graph(0)`, `ValueError`),
 - ii. ("directed graph", `nx.DiGraph([(1, 2)])`, `nx.NetworkXError`),
 - iii. ("self-loop", `nx.Graph([(1, 1), (1, 2)])`, `nx.NetworkXError`),
 - iv. ("multigraph", `nx.MultiGraph([(1, 2), (1, 2)])`, `nx.NetworkXError`),

I wanted to make the 'graphs' small / as simple as possible so that I could be sure it was correctly identifying the errors. (and I'm lazy).

For the next section, I was double-checking that my answers aligned with the `networkx` implementation. Because if they don't match, that would be a sign there is an issue with my code. Either in how it's filtering graphs or how it's calculating the spectral decomposition. I chose the empty graph because I can hand compute the values and easily verify everything works; it also makes for a good error handling case. Triangle and Peterson are both regular graphs, so I expect to see each node having the same score. For the disconnected graph, I wanted to do error handling, but I also wanted to compare the u_i values since I know that $\rho(C_3) = \rho(C_3 + P_3) = 2$, where ρ is the spectral radius. In the norm vectors for both graphs, I can see that the u_i values for the triangle are actually the same! Which logically makes sense because of what Subgraph centrality is measuring. I wanted to use the star graph to 1) compare it to the regular graphs and 2) make sure my intuition is right (that all the leaf nodes will have the same score). Finally, I used the Karate Club because I wanted a graph that was more irregular than all the other ones. I also thought it would be an interesting way to "measure" who is present in the most "subgraph communities" in the network; this would be someone who is deeply embedded in the local community. The results showed that Node 33 and 0 (the two club leaders) had the highest sg-centrality, which makes sense.

- i. "empty edges": `nx.empty_graph(4)`,
- ii. "triangle": `nx.complete_graph(3)`,
- iii. "Petersen": `nx.petersen_graph()`,
- iv. "disconnected": `nx.disjoint_union(nx.cycle_graph(3), nx.path_graph(3))`,
- v. "star": `nx.star_graph(5)`,
- vi. "karate club": `nx.karate_club_graph()`,

For the final section, I wanted to test the case for when e^λ causes an overflow. Clearly, for both my implementation and NetworkX's implementation, it overflows for the unnormalized case, and (after checking their docs) we both use a similar method for dealing with the overflow in the normalized case. I think people should (almost) always use the normalized score. Not only does the subgraph centrality score become incomputable for most graphs with a large spectral radius, but it also gives less meaning to the score when comparing graphs. The normalized score makes for a more intuitive (and computationally safer) metric, as comparisons can be made between different graphs. However, when you are specifically interested in the number of subgraphs that a node participates in, then you will use the unnormalized subgraph centrality.

One should note that for e^x as $x \rightarrow -\infty$, the values will get increasingly tiny. This is the correct behavior and shouldn't cause division by 0 for finite graphs. But values could get so small that rounding leads to the subgraph centrality score being 0 even though that is not possible for a non-isolated node.

4. Question 4.

- (a) Which department has the most prerequisites per course, on average? Say how you counted. Is this surprising to you?

The department with the highest Pre-Reqs is ANTH, with an average of 2.685714 pre-reqs per course. The department has 35 courses, so this is not an anomaly. I think it makes sense that ANTH has so many pre-reqs since, in a way, it is the science of human history/behavior, a subject that naturally builds upon itself. This knowledge cannot be derived from a understanding of the fundamentals but instead must be taught.

For each catalog course, I counted its prerequisites as its in-degree (the number of distinct linked course codes in its "Prerequisite(s)" line, excluding corequisites and unlinked text), then averaged those counts across all of a department's courses, including courses with zero prerequisites.

- (b) Is your graph actually a DAG? If not, find a few cycles, look those courses up in the catalog, and explain where the cycles come from. Then find the longest chain of prerequisites you can.

No, the resulting graph is not a DAG, and `nx.is_directed_acyclic_graph(G)` returns False because of 8 cycles: 6 self-loops and two 2-course cycles.

Self-loops (SPNS 1101, 1102, 2101, 2102, 3101, 3102). The catalog lists a placement-exam alternative in each prerequisite line, for example, "Placement in SPNS 1101 with a score of 1101" for SPNS 1101. The course code inside

that phrase is a hyperlink, so my scraper treated it as a course prerequisite and created an edge from the course to itself. SPNS 1101's only prerequisite is that placement phrase, which shows the self-loop is a scraping artifact and not a real requirement. A student places into the course by exam score, not by taking it first.

EDU 7140 $\leftrightarrow$ EDU 7150. Each course lists the other inside one branch of an "or" requirement:

EDU 7140 requires (EDU 7100; EDU 7130); EDU 7150 or (EDU 7207; 7218; 7225).

EDU 7150 requires (EDU 7130; EDU 7140) or (EDU 7207; 7218; 7219; 7225; 7226).

Either course can be taken first by using the other branch (EDU 7207/7218/7225), so there is no real circular dependency. The cycle appears because the graph draws an edge for every linked course and ignores the "or" structure.

SPNS 3602 $\leftrightarrow$ SPNS 3603. Each lists the other in a long "or" list (SPNS 3101 or 3102 or 3601 or the other course, or placement). Both courses are linguistics electives that are interchangeable as prerequisites for each other, and a student can enter either one through SPNS 3101 or 3102. This is the same "or" artifact. Common cause: the catalog's prerequisite lines are logical expressions with "or" and "and", and a graph with plain edges can't represent them. Every linked course becomes an edge, so "A or B" looks like "A and B". This also means prerequisite counts (and the longest chain) are inflated.

The longest chain has 10 courses (9 steps): MATH 1231 $\rightarrow$ MATH 1242 $\rightarrow$ PHYS 1191 $\rightarrow$ PHYS 1165 $\rightarrow$ EECE 2150 $\rightarrow$ EECE 3468 $\rightarrow$ EECE 5644 $\rightarrow$ CS 7140 $\rightarrow$ CS 6180 $\rightarrow$ DADS 7305.

Checking the catalog, only EECE 2150 $\rightarrow$ EECE 3468 is a mandatory requirement. In the other 8 links, the earlier course is one of several alternatives joined by "or" (for example, DADS 7305 accepts any one of 13 courses), and two links allow concurrent enrollment. Because my graph draws an edge for every linked course and ignores "or" versus "and", it treats alternatives as if all were required. So this chain shows the longest path through the catalog's links, not a sequence a student must complete. The real minimum number of courses needed to reach DADS 7305 is much shorter.

Though the power of graph theory, I know that cutting any "chordal edge" I can obtain the minimum spanning tree. We first removed the 6 self-loops (SPNS courses whose placement-exam phrase linked back to themselves), then removed every edge inside each remaining strongly connected component, which deleted both directions of the SPNS 3602 $\leftrightarrow$ 3603 and EDU 7140 $\leftrightarrow$ 7150 pairs. This left 5,670 edges and a true DAG, though it is conservative and may cut real chains through those courses, so the longest path is a lower bound.

(c) Does any of this change your answer to (c)?

Reevaluation for the average prereqs per department with the acyclic graph shows that ANTH still remains in the top with an average of 2.685714 prereqs per course. The other top departments have identical averages as well. This is expected, since only 10 of 5,680 edges were removed and all of them belonged to SPNS or EDU

courses.

- (d) The prerequisite network can answer questions about curricular dependency, such as which departments serve as foundations for others, because its edges are directed and encode “must come before.”

- i. Which courses are bottlenecks, meaning how many downstream courses become unreachable if a course isn’t offered?

The co-occurrence network can answer questions about topic similarity, such as which concepts are taught together across unrelated departments, because it is built from the content of course descriptions rather than from how courses are formally linked.

- i. Which topics link departments that share no prerequisite relationship, suggesting interdisciplinary overlap?

5. Question 5.

- (a) We randomized a copy of the network with `nx.double_edge_swap`, using $10 \times 16,630 = 166,300$ swaps and `max_tries` set to $100 \times$ that number. Every node kept its degree (the per-node degrees are identical), and 5.3% of the edges are still identical to the original, so the graph is well mixed.

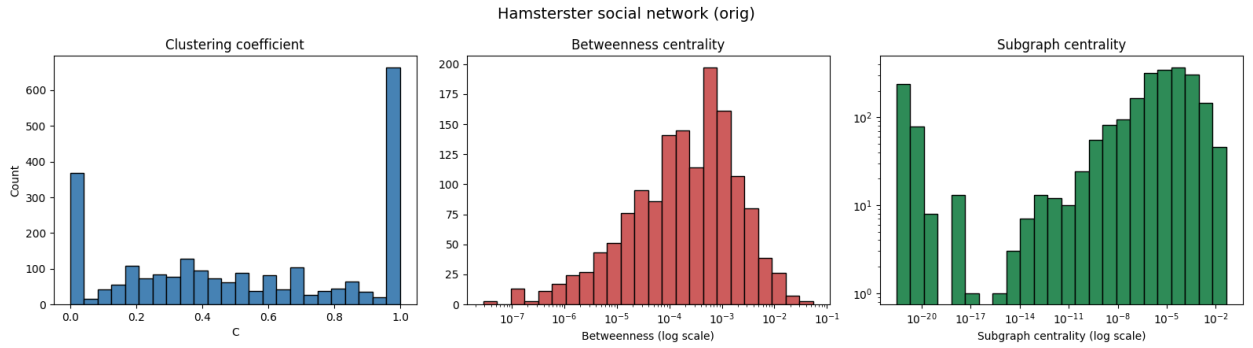


FIG. 1. Original network: clustering coefficient, betweenness centrality, and normalized subgraph centrality.

- i.

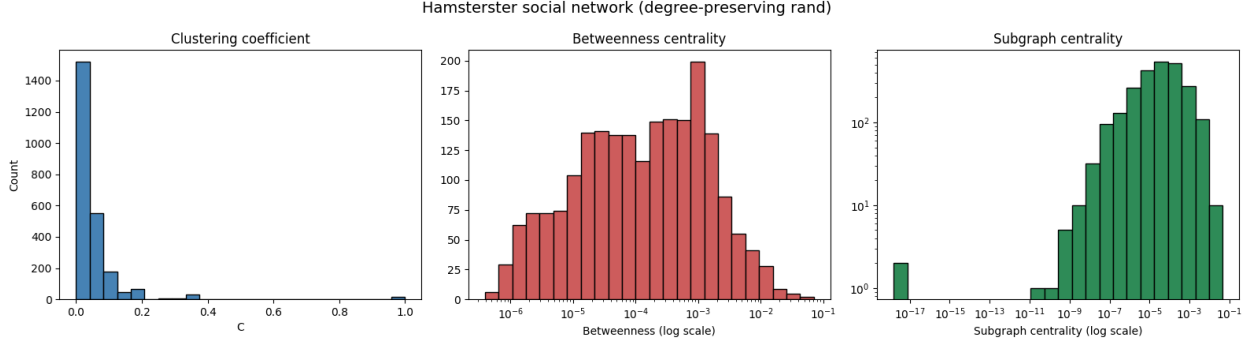


FIG. 2. Degree-preserving randomization, same statistics and bins.

- ii. *Clustering*. The mean is 0.538 and the distribution is bimodal. Of the 367 nodes with $C = 0$, 304 have degree one (clustering is zero by convention). The 653 nodes with $C = 1$ are mostly members of small tight groups.

Betweenness. 957 nodes (39.4%) have zero betweenness and are not shown on the log axis. The rest span about five orders of magnitude, with a single peak near 10^{-3} (mean 7.3×10^{-4} , max 0.055), so a few nodes carry far more shortest paths than the typical node.

Subgraph centrality. The bulk lies between 10^{-14} and 0.045. A separate group of 428 nodes lies below 10^{-15} , and 426 of them are outside the giant component. This group is an artifact of normalizing by e^ρ with $\rho = 50.0$, a value set by the giant component. The network has 148 components (giant component: 2000 nodes, 82.4%).

Clustering collapses to a mean of 0.047, with most nodes near zero. Only 15 nodes have $C = 1$. *Betweenness* has only 319 zeros (13.1%), a flatter distribution, a larger mean (8.8×10^{-4}), and a larger maximum (0.071). *Subgraph centrality* has one mode. Only the two nodes of a two-node component fall below 10^{-15} . The randomized graph has 2 components (giant: 2424 nodes), and $\rho = 41.4$.

- (b) Clustering is the clearest difference. The average falls from 0.538 to 0.047, so the triangles in the real network are not explained by its degree sequence. The dominant eigenvalue also drops (50.0 to 41.4), which is consistent with dense local groups raising ρ .

The null model does not preserve connectivity: it merges 148 components into 2. Several differences therefore come from connectivity and not from clustering, including the fraction of zero-betweenness nodes (39.4% to 13.1%), the shift in mean betweenness, and the whole low-score group in subgraph centrality. A comparison restricted to the giant component would separate these effects.

Comparing against a null model tells us which features need an explanation beyond the degree sequence (here, triangles and fragmentation) and which do not.

-
- [1] E. W. Weisstein. “Unswitchable Graph.” From *MathWorld—A Wolfram Web Resource*. <https://mathworld.wolfram.com/UnswitchableGraph.html> [Accessed: September 29, 2026].
 - [2] J. Kunegis. “KONECT: The Koblenz network collection.” In *Proceedings of the 22nd International Conference on World Wide Web Companion*, pp. 1343–1350, 2013.
 - [3] R. A. Rossi and N. K. Ahmed. “The Network Data Repository with interactive graph analytics and visualization.” In *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 29, no. 1, 2015.